W2D2

Created time	October 13, 2026 5:06 PM

Q1.

```
CREATE database IF NOT EXISTS britmart_lab;

USE britmart_lab;

CREATE TABLE customers (
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100) NOT NULL,
city VARCHAR(50) NOT NULL
);

CREATE TABLE orders (
order_id INT PRIMARY KEY,
customer_id INT NOT NULL,
amount_gbp DECIMAL(10,2) NOT NULL,
status VARCHAR(20) NOT NULL,
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id)
);
```

Q2.

```
INSERT INTO customers
(customer_id, customer_name, city)
VALUES
(1, 'Amelia Clarke', 'London'),
(2, 'Oliver Bennett', 'Manchester'),
(3, 'Aisha Khan', 'Leeds');

INSERT INTO orders
(order_id, customer_id, amount_gbp, status)
VALUES
(101, 1, 45.50, 'complete'),
(102, 2, 18.00, 'cancelled'),
(103, 1, 62.25, 'complete'),
(104, 3, 30.00, 'complete');

SELECT
o.order_id,
c.customer_name,
c.city,
o.amount_gbp,
o.status
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

Q3.

```
customer1 = {
"customer_id": 1,
"name": "Amelia Clarke",
"city": "London",
```

```
"email": "amelia@example.com",
"preferences": {
"newsletter": True,
"language": "en",
},
}

customer2 = {
"customer_id": 2,
"name": "Oliver Bennett",
"city": "Manchester",
"preferences": {
"newsletter": False,
"language": "en",
},
}

print(customer1)
print(customer2)
```

Q4.

C. Orders must reference valid customers and transaction consistency is important.

Q5.

A. Product records have different optional attributes. —- Document
B. Session ID must return session data very quickly. —— Key-value
C. Large event data is grouped by a known key and may contain many columns. —— column-based
D. Customer profile contains nested addresses and preferences. ——Document

Q6.

```
sessions = {
"SESSION-1001": {
"customer_id": 1,
"cart_count": 3,
},
"SESSION-1002": {
"customer_id": 2,
"cart_count": 1,
},
"SESSION-1003": {
"customer_id": 3,
"cart_count": 5,
},
}

print(sessions["SESSION-1002"])
```

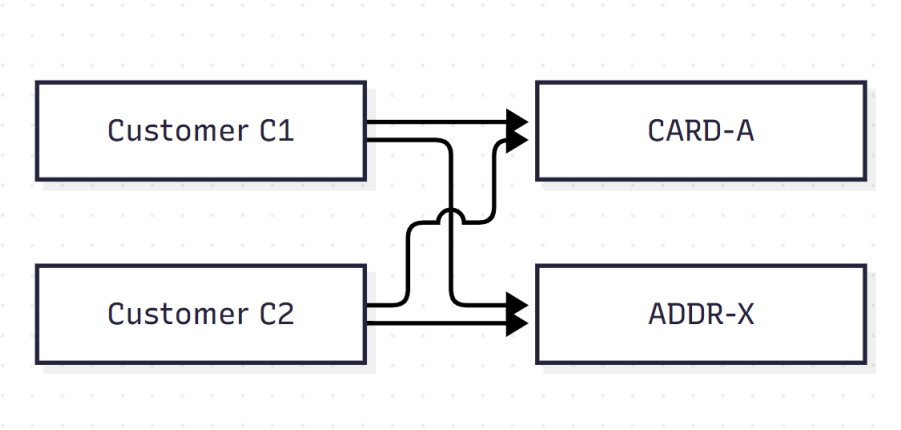
Q7.

```
products = [
{
"product_id": "P101",
"name": "Office Chair",
"colour": "Black",
"material": "Mesh",
},
{
"product_id": "P102",
```

"name": "Laptop Stand",
"colour": "Silver",
"height_adjustable": True,
},
{
"product_id": "P103",
"name": "Notebook Set",
"page_count": 200,
"paper_size": "A5",
},
]

Q8. A. Find a session directly from its session ID

Q9.



Q10.

A. An order and its payment should not be half-completed. —- Atomicity
B. A successful commit remains stored after a restart. ——- Durability
C. Two transactions should not see unsafe partial work from each other. ——Isolation
D. A transaction must leave the database in a valid state. ——Consistency

Q11.

Requirement	Choice	Reason
Payment update	ACID	Strong correctness is important
Social activity count	BASE	Eventual consistency is acceptable; counts can update after a short delay.
Order and stock update	ACID	Related changes must remain consistent
High availability with temporary inconsistency	BASE	Availability is prioritised

Q12. B. Temporary inconsistency may be accepted to support availability and scale

Q13.

Data	Choice	Reason
Payment transaction	Fixed	Required structure and correctness matter
Product catalogue	Flexible	Attributes can differ by product type
Order line	Fixed	Required fields and relationships are important
Event payload	Flexible	Optional fields can vary

Q14.

1. Customer orders with strong foreign-key relationships. — Relational
2. Product descriptions with many optional attributes. — Document
3. User session lookup by session ID. — Key-value
4. Fraud investigation based on many connected entities. — Graph

Q15

Risks include:

- required business fields may be missing; Unclear payment status
- relationships to orders may be difficult to enforce;
- payment reporting may become unreliable.

Q16. BritMart must store orders, customers, and payments with strong relationships and transaction correctness. Which is the strongest starting choice? D. Relational database

Q17

1. Relational model — table rows

In a relational database, both products can be stored in the same table. Attributes that do not apply to a product are set to Null.

Products table

product_id	name	price_gbp	colour	motor_count	height_min_cm	height_max_cm	page_count
P200	Smart Desk	450	White	2	70	120	NULL
P201	Notebook	5	NULL	NULL	NULL	NULL	200

2. Document model — separate documents

In a document database, each product can have its own set of fields.

Document 1 — Smart Desk

```
{
  "product_id": "P200",
  "name": "Smart Desk",
  "price_gbp": 450,
  "colour": "White",
  "motor_count": 2,
  "height_min_cm": 70,
  "height_max_cm": 120
}
```

Document 2 — Notebook

```
{
  "product_id": "P201",
  "name": "Notebook",
  "price_gbp": 5,
  "page_count": 200,
  "paper_size": "A5"
}
```

A document database is a more natural fit when products have significantly different optional attributes. A relational model can still be preferable when strong structure, constraints, and relationships are more important.

Q18.

```
relationships = [
("C1", "CARD-A"),
("C2", "CARD-A"),
("C1", "ADDR-X"),
("C2", "ADDR-X"),
("C3", "CARD-B"),
]

c1_values = []
c2_values = []

for customer, value in relationships:
if customer == "C1":
c1_values.append(value)
elif customer == "C2":
c2_values.append(value)

c1_set = set(c1_values)
c2_set = set(c2_values)

shared_values = c1_set.intersection(c2_set)

print("C1 values:", c1_values)
print("C2 values:", c2_values)
print("Shared values:", shared_values)
```

Q19.

System	Database model	Schema style	Consistency need	Main access pattern
Checkout orders	Relational (SQL)	Fixed schema	Strong consistency (ACID)	Create and update orders; retrieve order and payment details
Product catalogue	Document (NoSQL)	Flexible schema	Moderate consistency; changes should eventually propagate	Read and search product details by product ID, category, or attributes
Website sessions	Key-value (NoSQL)	Schema-light	Eventual consistency is often acceptable; session updates should be fast	Read and write session data by session ID
Fraud relationship investigation	Graph database	Flexible, relationship-oriented schema	Consistency for recorded relationships; timely updates are desirable	Traverse links between customers, cards, addresses, devices, and transactions

Q20.

NoSQL is not always faster. Performance depends on the data structure, query patterns, workload, and database design.

- NoSQL advantages: Flexible schemas, horizontal scalability, and fast access for certain workloads.
- SQL advantages: Strong transactional consistency, structured relationships, and powerful queries.
- Trade-off: Some NoSQL systems prioritize availability and scalability over immediate consistency, while others support strong consistency.

BritMart should use the right database for each workload. Relational databases are suitable for checkout orders and payments, document databases for flexible product catalogues, key-value databases for website sessions, and graph databases for fraud investigations.